⏱️ DAY 5: COMPLETE 6-HOUR MASTER TRAINER PLAYBOOK

Generative AI & Large Language Models on Intel HPC: Habana Gaudi & OpenVINO

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Daily Duration:** 6 Full Hours (Session 1: Morning 3h + Session 2: Afternoon 3h)
Official Syllabus Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)
Day 5 Complete Terminology Glossary: `00_DAY_05_MASTER_TERMINOLOGY_AND_CONCEPT_GLOSSARY.md`
Day 5 Hands-On Lab Runbook: `02_DAY_05_HANDS_ON_LAB_AND_COMMANDS.md`

🎯 1. Official Syllabus Scope (Page 2 of PDF — Day 5 Capstone)

The official curriculum specifies **strictly 4 core topics for Day 5**:

1. **Introduction to Generative AI & LLMs on HPC**
2. **Intel hardware for LLMs (Xeon, Habana Gaudi)**
3. **Optimizing LLM inference with OpenVINO Toolkit**
4. **Fine-tuning LLMs on HPC**

> 📌 **IMPORTANT**
> **Curriculum Boundary Discipline:** Day 5 is the grand capstone synthesis. We apply all Linux, oneAPI, OpenMP, MPI, and Slurm concepts mastered in Days 1 to 4 directly to the most demanding computational frontier: Generative AI and Large Language Models. Keep the focus razor-sharp on model ingestion, IR graph export, NNCF INT4 quantization, real-time TTFT/TPS latency benchmarks, distributed LoRA fine-tuning, and production API serving.

⏰ 2. Complete 6-Hour Dual-Session Master Daily Schedule

```
+-------------------------------------------------------------------------------------+
|                    DAY 5 MASTER 6-HOUR SCHEDULE (09:30 AM - 04:30 PM)               |
+-------------------------------------------------------------------------------------+
| 👷 SESSION 1: MORNING THEORY & ARCHITECTURE MASTERCLASS (09:30 AM - 12:30 PM | 3 Hours)  |
|    "Transformer Silicon Physics → Xeon AMX & Gaudi RoCE → OpenVINO & LoRA Math"     |
+-----------------------+-------------------------------------------------------------+
|  09:30 - 10:15 (45m)  | PART 1: Introduction to Generative AI & LLMs on HPC         |
|                       |        (Transformer Silicon Physics, Memory Wall, KV-Cache Math)  |
+-----------------------+-------------------------------------------------------------+
|  10:15 - 11:00 (45m)  | PART 2: Intel Hardware for LLMs: Xeon AMX vs Habana Gaudi   |
|                       |        (AMX Tile Registers vs 24x 100GbE RoCE On-Die Networking)  |
+-----------------------+-------------------------------------------------------------+
|  11:00 - 11:10 (10m)  | ☕ REJUVENATION BREAK (10 Minutes)                           |
+-----------------------+-------------------------------------------------------------+
|  11:10 - 11:45 (35m)  | PART 3: Optimizing LLM Inference with Intel OpenVINO Toolkit       |
|                       |        (IR Graph Optimization, NNCF INT4 Weight Quantization)     |
+-----------------------+-------------------------------------------------------------+
|  11:45 - 12:15 (30m)  | PART 4: Fine-Tuning LLMs on HPC via Slurm & PEFT/LoRA       |
|                       |        (Low-Rank Adapters Math, Distributed Gradient Synchronization) |
+-----------------------+-------------------------------------------------------------+
|  12:15 - 12:30 (15m)  | PART 5: Morning Review, 5 Rapid Check Questions & Lunch Teaser    |
+-------------------------------------------------------------------------------------+
| 🍱 LUNCH & INTERMISSION BREAK (12:30 PM - 01:30 PM | 60 Minutes)                    |
|    HuggingFace cache pre-population, OpenVINO CPU runtime initialization            |
+-------------------------------------------------------------------------------------+
| 💻 SESSION 2: AFTERNOON HANDS-ON SUPERCOMPUTER LAB (01:30 PM - 04:30 PM | 3 Hours)        |
|    "OpenVINO IR Export → INT4 Quantization → Streaming TTFT/TPS → LoRA Cluster → 25 MCQs"|
+-----------------------+-------------------------------------------------------------+
|  01:30 - 02:15 (45m)  | STAGE 0 & 1: Model Ingestion & OpenVINO IR Conversion       |
|                       |        (ov.convert_model, inspecting .xml graph topology & .bin)  |
+-----------------------+-------------------------------------------------------------+
|  02:15 - 03:00 (45m)  | STAGE 2: NNCF INT4 Weight-Only Quantization Benchmark       |
|                       |        (Compressing FP16 to INT4, verifying 72% RAM reduction)   |
+-----------------------+-------------------------------------------------------------+
|  03:00 - 03:45 (45m)  | STAGE 3: Real-Time Streaming LLM Latency Benchmark          |
|                       |        (Measuring TTFT Time-to-First-Token & Tokens/Sec (TPS))    |
+-----------------------+-------------------------------------------------------------+
|  03:45 - 04:15 (30m)  | STAGE 4 & 5: Slurm Distributed LoRA Fine-Tuning & FastAPI Serving |
|                       |        (Authoring #SBATCH LoRA training & deploying async REST API)|
+-----------------------+-------------------------------------------------------------+
|  04:15 - 04:30 (15m)  | GRAND CAPSTONE ASSESSMENT: 25 Evaluated MCQs & Course Graduation  |
|                       |        (22-Point Mastery Checklist, 25 MCQs, Certificate Sign-Off)|
+-------------------------------------------------------------------------------------+
```

🎙️ 3. Line-by-Line Minute-by-Minute Trainer Delivery Script (Session 1: Morning)

PART 1: Introduction to Generative AI & LLMs on HPC (09:30 - 10:15 | 45 Mins)

- **Slide Alignment:** Day 5 Slide Deck (Slides 1–5: Transformer Silicon Physics & Bottlenecks).
- **Concept:** Auto-regressive decoding, Prefill vs Decode phase, and KV-Cache memory consumption.
- **Trainer Script (Hinglish/English):**

> "Welcome to Day 5 — the grand capstone of our supercomputing journey! Over the last 4 days, you mastered Linux, oneAPI, OpenMP multithreading, MPI message passing, Slurm job scheduling, and VTune microarchitecture profiling. Today, we bring every single skill together to solve the most demanding problem in modern computing: deploying, optimizing, and fine-tuning Large Language Models on enterprise Intel HPC hardware. Let's start with a brutal physical truth: Why do LLMs break ordinary computers? When generating text with an LLM, generating token #100 requires reading ALL previous 99 tokens from memory! This divides text generation into two distinct phases:
>
> 1. **Prefill Phase (Prompt Processing):** Compute-bound. All input prompt tokens are ingested simultaneously in a massive matrix multiplication ($GEMM$).
> 2. **Decode Phase (Token by Token Generation):** Memory-bandwidth bound! To emit just ONE single new word, the processor must stream all 7 Billion weights from RAM into registers! If your memory bandwidth is slow, your model generates 1 word per second. Furthermore, look at the **KV-Cache (Key-Value Cache)**. Storing attention matrices across layers for long context windows consumes gigabytes of memory per user! In HPC, we defeat the memory wall using high-bandwidth memory (HBM), hardware matrix accelerators, and model quantization."

- **Whiteboard Sketch 1:**

```
[ Prefill Phase ] (Prompt Ingestion)       [ Decode Phase ] (Token Generation)
Matrix x Matrix = COMPUTE-BOUND            Vector x Matrix = MEMORY-BANDWIDTH BOUND
All prompt tokens processed together       Must load ALL 7B weights to output 1 token!
(High Arithmetic Intensity: FLOPs/Byte)    (Low Arithmetic Intensity -> Bandwidth Starvation)
```

PART 2: Intel Hardware for LLMs: Xeon AMX vs Habana Gaudi (10:15 - 11:00 | 45 Mins)

- **Slide Alignment:** Day 5 Slide Deck (Slides 6–8: Intel Silicon Engines for LLMs).
- **Concept:** CPU matrix engines (AMX) vs dedicated AI ASICs with on-chip networking.
- **Trainer Script:**

> "Most developers think you cannot run LLMs without buying a 50-lakh rupee GPU cluster. That is completely false. Look at 4th Gen Intel Xeon Scalable processors (Sapphire Rapids). They feature **Intel AMX (Advanced Matrix Extensions)**. AMX is a built-in systolic array matrix accelerator sitting directly on each CPU core! It operates on 16x16 tile registers (TMM0 to TMM7), executing 2048 INT8 operations or 1024 BF16 operations per clock cycle. For enterprise chatbots, code generation, and medium-scale LLM inference, Xeon CPUs with AMX give real-time responses at a fraction of GPU infrastructure cost. Now, what if you need to train or fine-tune a massive 70-Billion parameter model? Here is where **Habana Gaudi** dominates. Gaudi is not a GPU that was modified for AI. It is an application-specific processor designed purely for deep learning. It has 24 Tensor Processor Cores (TPCs) and 96GB to 128GB of High-Bandwidth Memory (HBM). But its absolute superpower is this: **24 integrated 100GbE RoCE (RDMA over Converged Ethernet) ports directly on the silicon die!** In typical clusters, GPUs must route network data through external PCIe slots and switches, causing bottlenecks. Gaudi connects node-to-node directly through its silicon die, allowing near-linear scaling across thousands of accelerators."

☕ BREAK (11:00 - 11:10 | 10 Minutes)

PART 3: Optimizing LLM Inference with Intel OpenVINO (11:10 - 11:45 | 35 Mins)

- **Slide Alignment:** Day 5 Slide Deck (Slides 9–10: OpenVINO Compression & Graph Optimization).
- **Concept:** Compiling PyTorch graphs to Intermediate Representation (IR) and INT4 quantization.
- **Trainer Script:**

> "How do we compress a 7-Billion parameter model so it runs at 30 tokens per second on a CPU server? We use **Intel OpenVINO Toolkit**. The pipeline has 4 steps:
>
> 1. **Model Ingestion:** Loading pre-trained HuggingFace weights (PyTorch/Safetensors).
> 2. **IR Conversion:** Compiling to OpenVINO Intermediate Representation (`.xml` graph structure + `.bin` weights).
> 3. **NNCF Weight-Only Quantization:** Compressing 16-bit floating point weights down to **INT4 (4-bit integers)**!
> 4. **Hardware Execution:** Automatically mapping matrix computations to Intel AMX on CPUs or Xe-Cores on GPUs. Look at the math: In FP16, a 7B model requires $7 \times 2 = 14$ GB of DRAM bandwidth per token. In INT4, it requires only $7 \times 0.5 = 3.5$ GB! You just cut memory bandwidth traffic by **75%**, doubling your token generation speed!"

PART 4: Fine-Tuning LLMs on HPC via Slurm & LoRA (11:45 - 12:15 | 30 Mins)

- **Slide Alignment:** Day 5 Slide Deck (Slides 11–14: Distributed PEFT & LoRA Fine-Tuning).
- **Concept:** Low-Rank Adaptation mathematical mechanics and multi-node batch execution.
- **Trainer Script:**

> "To adapt an LLM for GLA University course Q&A, do we need to retrain all 7 Billion parameters? NO! We use **LoRA (Low-Rank Adaptation)**. Instead of modifying the full weight matrix $W_0 \in \mathbb{R}^{d \times k}$, we freeze W_0 and inject two low-rank decomposition matrices $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, where rank $r \ll d$ (e.g., $r = 8$ or $r = 16$). The forward pass becomes:
>
> $$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} B A x$$
>
> Trainable parameters drop from 7,000,000,000 down to 20,000,000 (a 99.7% reduction!). Memory consumption during backpropagation plummets, allowing us to fine-tune on standard HPC compute nodes using Slurm batch scripts!"

PART 5: Morning Review, 5 Rapid Check Questions & Lunch Teaser (12:15 - 12:30 | 15 Mins)

[12:15 - 12:25] 5-Question Quick Classroom Check

1. *Q1: Why is the Decode phase of LLM text generation memory-bandwidth bound?*
   Answer: Emitting each single new token requires loading all model weights from RAM into registers; low arithmetic intensity causes execution units to starve for memory data.
2. *Q2: What is the hardware function of Intel AMX in 4th/5th Gen Xeon processors?*
   Answer: Dedicated matrix tile acceleration (TMUL) with 8 hardware registers (`tmm0` – `tmm7`) computing 2048 INT8 or 1024 BF16 ops per clock cycle.
3. *Q3: What unique networking advantage is integrated directly on the Habana Gaudi silicon die?*
   Answer: 24 integrated 100GbE RoCE v2 ports directly on-chip, enabling direct chip-to-chip scaling without external PCIe switches.

4. *Q4: What memory footprint reduction is achieved by OpenVINO NNCF INT4 weight-only quantization?*
   Answer: A ~70% to 75% reduction in model weight memory footprint, dropping a 7B model from ~14 GB (FP16) down to ~3.5-4 GB (INT4).
5. *Q5: In LoRA fine-tuning, how do low-rank matrices A and B reduce GPU memory requirements?*
   Answer: By freezing >99% of original base model weights and only computing and storing optimizer states and gradients for rank r matrices (e.g., $r = 8$ or 16).

[12:25 - 12:30] Lunch Break Announcement & Afternoon Lab Setup

- **What Trainer Shivam Says:**

> "Incredible achievement this morning! We have covered the physics of Generative AI, Xeon AMX matrix engines, Habana Gaudi RoCE networking, OpenVINO INT4 compression, and LoRA fine-tuning. We now take a 1-hour lunch break until 01:30 PM. In our final afternoon session, we will convert models to OpenVINO IR, quantize them to INT4, measure real streaming tokens/second, deploy a FastAPI serving endpoint, complete our 25-MCQ capstone assessment, and sign off on your course completion. See you at 01:30 PM sharp!"

🍱 INTERMISSION: Lunch & Cluster Preflight (12:30 PM - 01:30 PM | 60 Mins)

- **Trainer System Checklist during break:**
  1. Verify OpenVINO and NNCF Python packages (`python3 -c "import openvino, nncf; print('OK')"`).
  2. Confirm synthetic test model weights are cached locally to avoid downloading over slow internet.
  3. Verify FastAPI and Uvicorn packages for the serving demonstration.
  4. Project Stage 0 preflight verification on the main lecture screen.

💻 4. SESSION 2: Afternoon Hands-On Supercomputer Lab Playbook (01:30 PM - 04:30 PM | 3 Hours)

Reference Runbook: `02_DAY_05_HANDS_ON_LAB_AND_COMMANDS.md`
Notebook Reference: `Day_05_Generative_AI_LLMs_OpenVINO_Gaudi_HPC_Master_Lab.ipynb`

```
+-------------------------------------------------------------------------------+
|                   SESSION 2: 3-HOUR HANDS-ON LAB EXECUTION PHASES             |
+-------------------------------------------------------------------------------+
| 01:30 - 02:15 (45m) | Phase 1: OpenVINO Model Ingestion & IR Conversion (Stages 0 & 1)   |
| 02:15 - 03:00 (45m) | Phase 2: NNCF INT4 Weight-Only Quantization Benchmark (Stage 2)    |
| 03:00 - 03:45 (45m) | Phase 3: Real-Time Streaming LLM Latency Benchmark (Stage 3)       |
| 03:45 - 04:15 (30m) | Phase 4: Slurm Distributed LoRA Fine-Tuning & FastAPI Serving (St 4)|
| 04:15 - 04:30 (15m) | Phase 5: Capstone Assessment, 25 MCQs & Course Graduation          |
+-------------------------------------------------------------------------------+
```

[01:30 - 02:15] Phase 1: OpenVINO Model Ingestion & IR Conversion (Stages 0 & 1)

- **Trainer Instruction (To Class):**

> "Welcome back to our final afternoon lab! Open your terminal or the Day 5 Google Colab notebook. In Stage 1, we ingest a Transformer model and export it to OpenVINO Intermediate Representation (IR). Look at cell 3: `ov_model = ov.convert_model(model, example_input=inputs)`. OpenVINO compiles the PyTorch compute graph into two files:
> - `model.xml`: XML topology describing layers, tensor dimensions, and fused activations.
> - `model.bin`: Binary weights stored contiguously for direct memory mapping (`mmap`). Inspect the generated XML file with `head -n 30 model.xml`."

[02:15 - 03:00] Phase 2: NNCF INT4 Weight-Only Quantization Benchmark (Stage 2)

- **Trainer Instruction (To Class):**

> "Now look at Stage 2: `nncf.compress_weights()`. We apply Asymmetric INT4 Weight-Only Quantization: `compressed_model = nncf.compress_weights(ov_model, mode=nncf.CompressWeightsMode.INT4_ASYM)`. Look at your disk and RAM footprint:
> - Uncompressed FP16 model: ~2.5 GB.
> - Compressed INT4 model: **~710 MB! (A 72% footprint reduction!)** Save it as `model_int4.xml` and `model_int4.bin`."

[03:00 - 03:45] Phase 3: Real-Time Streaming LLM Latency Benchmark (Stage 3)

- **Trainer Instruction (To Class):**

> "In Stage 3, we measure real hardware performance:
> 1. **TTFT (Time-To-First-Token):** Milliseconds taken to process prompt and emit token #1.
> 2. **TPS (Tokens-Per-Second):** Continuous text generation throughput during decode. Run cell 8 in the notebook. Compare FP16 vs INT4 on CPU: Notice that because INT4 cuts memory bus bandwidth by 72%, **Tokens-Per-Second jumps from 11 TPS up to 28 TPS!** That is the power of hardware-aware AI optimization."

[03:45 - 04:15] Phase 4: Slurm Distributed LoRA Fine-Tuning & FastAPI Serving (Stage 4)

- **Trainer Instruction (To Class):**

> "In Stage 4, inspect our distributed LoRA fine-tuning submission script `slurm_lora_finetune.sh`. Notice `#SBATCH --partition=gaudi` and `srun python3 finetune_lora.py`. Next, launch the low-latency FastAPI inference microservice: `python3 code/llm_fastapi_server.py`. In another terminal, query the endpoint: `curl -X POST http://localhost:8000/api/generate -H 'Content-Type: application/json' -d '{\"prompt\": \"What is HPC?\"}'`. You have built a complete, production-grade, hardware-accelerated AI serving microservice!"

[04:15 - 04:30] Phase 5: Capstone Assessment, 25 MCQs & Course Graduation

- **Trainer Activity:**
  1. Open `03_DAY_05_STUDENT_CHEATSHEET_AND_MCQS.md`.
  2. Review the 25 Evaluated Capstone MCQs covering Generative AI, AMX, Gaudi, OpenVINO, and LoRA.
  3. Walk through the final 22-point Day 5 Mastery Checklist.
  4. **Valedictory Closing Speech:**

> "Congratulations to each of the 60 students of GLA University! Over 5 intensive days and 30 full hours of training, you have mastered Linux internals, shell automation, SSH keys, Intel oneAPI compilers, OpenMP multithreading, MPI message passing, oneDAL machine learning, oneDNN deep learning, Slurm cluster job scheduling, VTune microarchitecture profiling, and Generative AI deployment. You are now prepared to build and deploy high-performance applications on the world's most powerful supercomputing clusters. Thank you and best wishes for your supercomputing careers!"

🛠️ 5. Top 5 Student Gotchas & 30-Second Rapid Fixes

Issue Encountered	Why It Happened	30-Second Trainer Fix
`ModuleNotFoundError: No module named 'openvino'`	Running inside Python environment without OpenVINO	Run `pip install openvino nncf transformers fastapi uvicorn`
INT4 Quantization crashes with out of memory	Quantizing full 70B model in a 16GB system	Use synthetic/small test model (Qwen-0.5B / TinyLlama) for classroom drills
LLM generation hangs during streaming	Streamer callback buffer not flushed	Ensure `streamer.end()` or standard generator yield loop is properly closed
FastAPI server fails with `Address already in use: 8000`	Previous uvicorn server process still alive	Kill previous process: `fuser -k 8000/tcp` or use alternate port `--port 8080`
Slurm LoRA job fails: `CUDA/Gaudi device not found`	Script running on CPU head node instead of compute node	Submit job via `sbatch` targeting the allocated accelerator partition